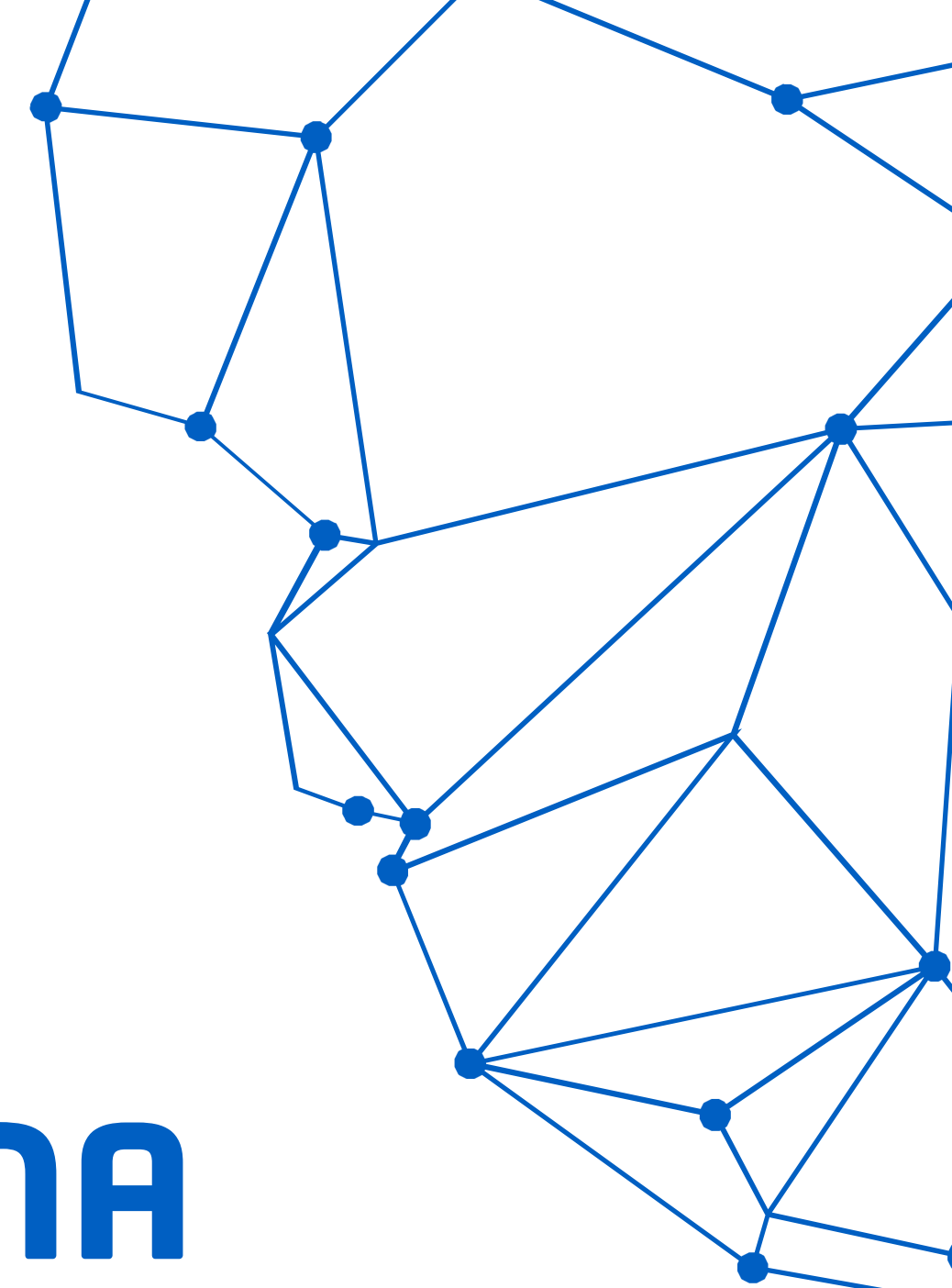
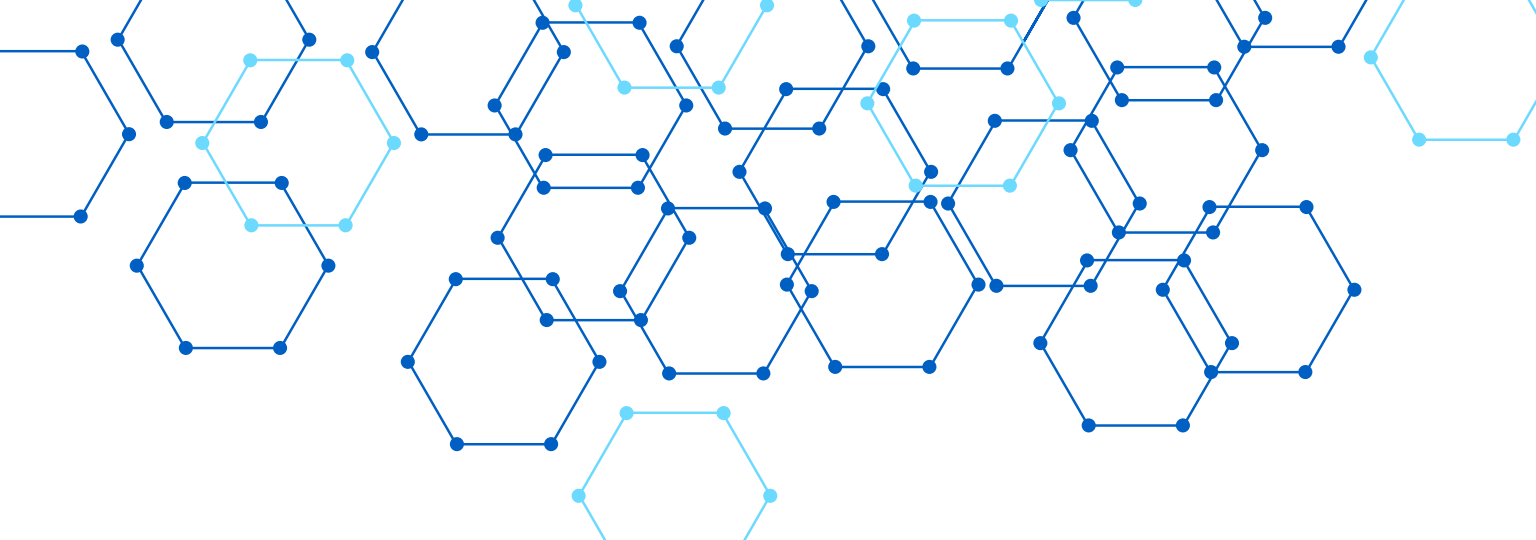
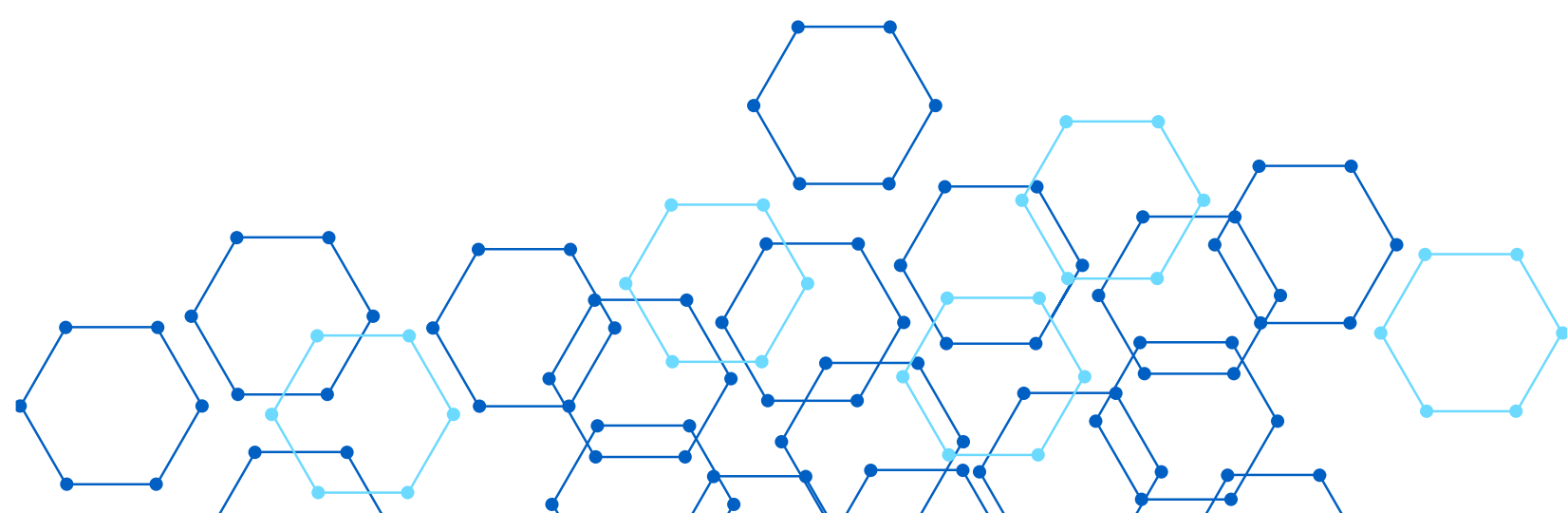




ESTRUCTURA DOCUMENTAL DEL PROYECTO LABORATORIO_SENA

Plantilla de documentación basada en SRS | Expositor: Juan
Gomez | 3 de julio de 2026



¿QUÉ ES UN SRS?

SRS significa Software Requirements Specification: un documento que reúne toda la información necesaria para desarrollar software.

- Organiza requisitos, arquitectura, datos, diseño y operación
 - Facilita que todo el equipo trabaje con la misma información
 - Establece una base común antes de comenzar el desarrollo
- "Laboratorio_sena toma estas partes del SRS y las organiza mediante carpetas estructuradas."



ORGANIZACIÓN GENERAL DEL REPOSITORIO



Objetivos y contexto: carpetas 01-context (alcance, glosario, visión general, límites del sistema) y 03-product (visión del producto, roadmap, backlog).



Requisitos y dominio: carpetas 02-domain (entidades, reglas del negocio, mapa de dominio) y 04-requirements (requisitos funcionales, no funcionales, historias de usuario).



Arquitectura y diseño: carpetas 05-architecture, 07-api, 08-uml y 09-microservices. Documentan decisiones técnicas, contratos API, diagramas y patrones de comunicación.

CARPETA 00-GOVERNANCE



Define las reglas y estándares del proyecto

- Normas generales de documentación
- Seguridad y control de acceso
- Convenciones Git para el equipo
- Definition of Ready (DoR)
- Definition of Done (DoD)

Importancia para el equipo

- Documentación de microservicios
- Establece estándares comunes para todos
- Evita inconsistencias en la documentación
- Garantiza que todos trabajen bajo las mismas reglas desde el inicio del proyecto



CARPETAS 01-CONTEXT Y 03-PRODUCT



01-context: Comprensión del entorno del sistema

- Alcance del proyecto
- Glosario de términos
- Contexto del sistema
- Visión general
- Límites del sistema



03-product: Planificación del producto a desarrollar

- Visión del producto
- Roadmap de entregas
- Backlog inicial del proyecto

CARPETAS 02-DOMAIN Y 04-REQUIREMENTS



02-domain: Entender el negocio

- Entidades del sistema
- Reglas del negocio
- Eventos del dominio
- Mapa de dominio

Define el contexto y la lógica del negocio antes de codificar.



04-requirements: Definir lo que debe hacer el sistema

- Requisitos funcionales
- Requisitos no funcionales
- Historias de usuario
- Matriz de trazabilidad

Garantiza que todos los requisitos estén documentados y rastreables.

CARPETAS 05-ARCHITECTURE Y 06-DATA



05-architecture — Base técnica del sistema

Visión arquitectónica del proyecto

Registro de decisiones arquitectónicas (ADRs)

Estrategia de despliegue e infraestructura

Aspectos transversales: seguridad, rendimiento, escalabilidad



06-data — Gestión de datos del proyecto

Diccionario de datos con definiciones y tipos

Modelo de datos: entidades y relaciones

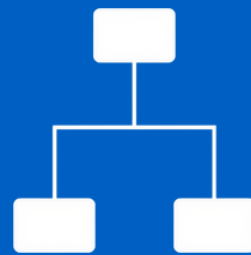
Estrategia de migración de datos

Estandariza cómo se almacena y accede a la información

CARPETAS 07-API, 08-UML Y 09-MICROSERVICES



07-api: Contratos API, autenticación, estándares y especificaciones OpenAPI para la documentación de interfaces del sistema.



08-uml: Diagramas UML, diagramas fuente y exportaciones que representan visualmente la estructura y comportamiento del software.



09-microservices: Catálogo de servicios, eventos, contratos, patrones de comunicación y runbooks operativos de cada microservicio.

CARPETAS 10-DEVOPS, 11-QUALITY Y 13-OPERATIONS



10-devops: CI/CD, gestión de entornos, configuración local y estrategias de despliegue continuo para automatizar la entrega del software.



11-quality: Estrategia de pruebas, revisión de código y estándares de calidad que garantizan la fiabilidad del sistema antes del despliegue.

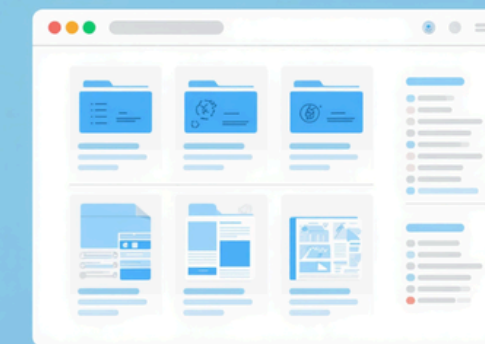


13-operations: Respaldos, recuperación ante fallos, gestión de incidentes y monitoreo continuo para asegurar la operación estable del sistema.

CARPETAS DE APOYO AL DESARROLLO



12-ux-ui: Wireframes y prototipos de interfaz, flujos de navegación, guías de diseño visual y componentes de UI para el sistema.



14-training y 15-project-control: Manuales de usuario, onboarding, gestión de riesgos, dependencias, backlog técnico y preguntas abiertas del proyecto.



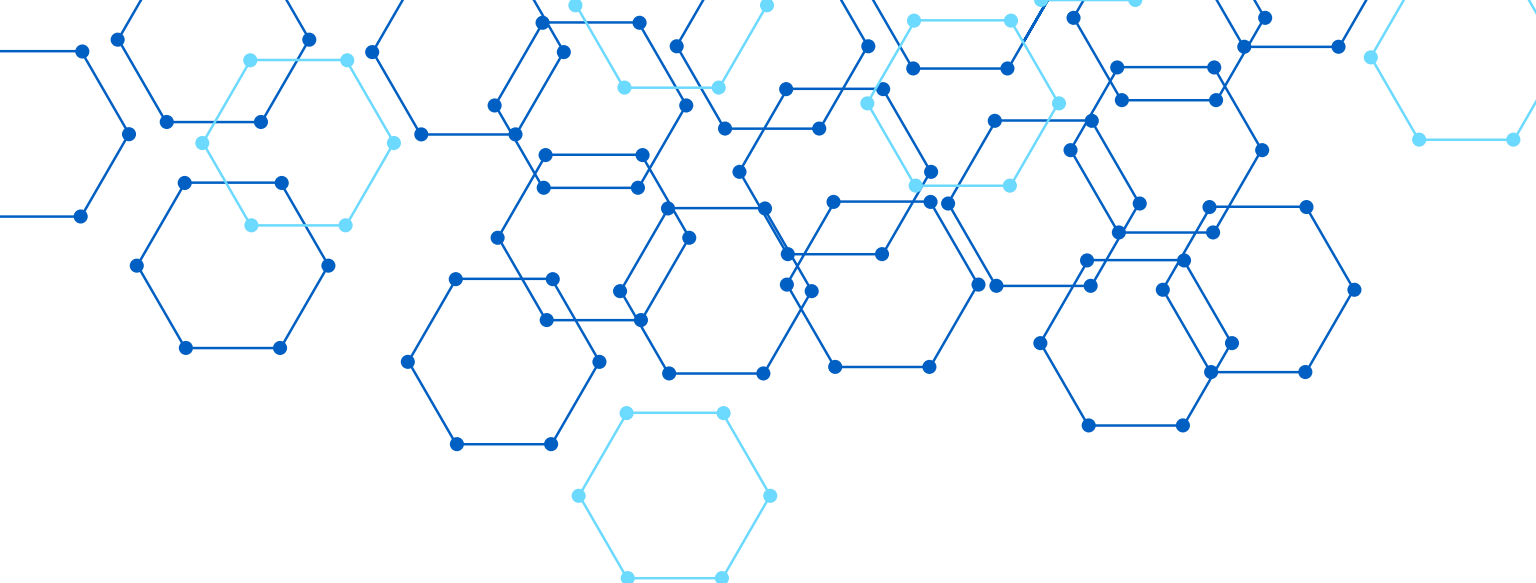
99-archive y assets: Documentación antigua o reemplazada en archivo histórico; imágenes, logotipos y recursos gráficos compartidos del repositorio.



CARPETAS CON REGLAS ESPECIALES



- Documentation-rules: estándares para redactar y estructurar documentos del proyecto.
 - Git-conventions: normas de ramas, commits y flujo de trabajo en el repositorio.
 - Security-rules: lineamientos de seguridad aplicables a todo el equipo.
-
- Definition-of-Ready y Definition-of-Done: criterios claros para iniciar y cerrar tareas.
 - Architectural decisions: registro de decisiones técnicas y su justificación.
 - Microservices-documentation: guías para documentar cada microservicio de forma uniforme.



CONCLUSIONES

"Una buena organización documental es tan importante como el código del software."
Juan Gomez · 3 de julio de 2026

